Intermediate (swap in a structured parser) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

You're going to swap the last piece of the chain: instead of "just give me text back," you'll say "give me back an object that matches this exact shape."

The shape is a small Pydantic class — the same kind you wrote in Doc04's structured-output exercise. Then you tell the model, "always answer in this shape."

- A Pydantic model: `class Person(BaseModel):` with `name: str` and `age: int`
- Attach it: `structured_model = ChatOpenAI(model="gpt-4o-mini", temperature=0).with_structured_output(Person)`
- Run it: `structured_model.invoke("Extract: Maria is 34 years old.")`

Then ask it something that has **no** person in it, and look closely at what comes back.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

`.with_structured_output(Person)` returns a new runnable whose output is already a `Person`, not text. With `gpt-4o-mini` it uses the same constrained decoding as Doc04's `.parse()` — the "shape during" method from the Output parsers topic.

That means a text with no person in it does **not** raise an error: the model **must** fill the form, so with `name: str` and `age: int` it invents something (`''` and `0`, for example). The fix is Doc04's rule again — `str | None` and `int | None` — so "nothing found" becomes a real answer.

A `ValidationError` is still possible (the reply fails the model in a rare case), so catch it by name.

Difference between Basic and Intermediate: Basic sees the invented values. Intermediate explains why they appear, and fixes them with `| None` fields.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define Person: name (text), age (number)
build a model that always answers in the Person shape
ask about Maria -> print the Person
ask about the sky -> print what comes back (no error!)
```

```
# structured_output_practice.py — Intermediate section
from pydantic import BaseModel
from langchain_openai import ChatOpenAI


class Person(BaseModel):
    name: str
    age: int


model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
structured_model = model.with_structured_output(Person)
```

Expected output if you run just this: nothing yet — add the two `.invoke()` calls and print both results.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

```
class Person: name: str | None, age: int | None

function extract_person(text) -> Person | None:
    try: person = structured_model.invoke(text)
    except ValidationError: return None
    if name is None and age is None: return None
    return person
```

This is the same shape as Doc04's `extract_info()` — only the call changed from `client.beta.chat.completions.parse(...)` to `.invoke(...)`. Write it, run it on both texts, then compare against the Solution.

Difference between Basic and Intermediate: Basic calls the structured model inline. Intermediate wraps it in `extract_person()`, which returns a checked `Person` or `None` — the caller has exactly two cases.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution